2025

Case Study: Evaluation of Technology

INTRODUCTION TO MOBILE APPLICATIONS DEVELOPMENT - CSP2108.1

DON MENUWAN KALANA RANAWEERA - 10685298

2/11/2025

Table of Contents

1. Application Overview

The Nulldle application is a mobile word puzzle game inspired by Wordle. The player's objective is to guess a hidden five letter word within six attempts. The app provides color-coded feedback after each guess for each letter: green for correct letters in the correct position, yellow for correct letters in the wrong position, and grey for letters not in the word. The provided Nulldle app had several design and functionality flaws. Mainly, Game logic and UI were tightly coupled making codes difficult to maintain and update. Also, keyboard color feedback had bugs causing it to not work. Functionally, the app allowed only five guesses instead of the required six guesses and repeated words were not checked correctly.

To fix these issues, the entire project was refactored and improved. The new version of the application follows the MVVM (Model-View-ViewModel) design pattern making it easier to maintain and update. Then, the keyboard color code updating bug and incorrect guessing limit problem was fixed. Additional improvements included adding word validation and preventing duplicate guesses. A statistic win, losses and incorrect guesses tracker introduced to improve the application. These changes not only made the app function correctly but also improved the user interface and the experience.

2. Changelog

Fixes

Location	Description of Change	Reason for Change
game_screen.dart	Corrected color feedback logic for the keyboard and for tiles	Previous logic showed wrong colors
game_view_model.dart	Added new if condition to prevent from entering repeated same guessing word.	Earlier code allowed same word multiple times
game_model.dart	Updated maxGuesses from 5 to 6.	To match with functional specifications by preventing the game stop after 5 guesses instead of 6.
game_screen.dart	Added a null check and used vm.isLoading and if not loaded show a progress indicator	To fix not Initialized error showed on screen

game_screen.dart, home_screen.dart	Wrapped the body in a Safe area.	To avoid layout touching edges of the screen.
game_screen.dart	Wrapped keyboard key containers in a flexible.	To avoid overflowing on some devices.
game_screen.dart	Changed win message close button front size to 24.	To improve the UI (earlier font size was 64 for win message which is inconsistent).

Refactor

Location	**Description of Change**	**Reason for Change**
game_model.dart	Created a GameModel to store game data	Centralized data structure for game state
game_screen.dart	Simplified UI logics stored here separated from logics	Cleaner code easy to make UI changes
home_screen.dart	Separated home screen from main	Cleaner navigation and structure
game_view_model.dart	Moved game state logic from UI to ViewModel	Separation of concerns, cleaner code, easier maintenance
main.dart	Initialized ViewModel using Provider	Flutter state management pattern

Feature Added

Location	**Description of Change**	**Reason for Change**
game_model.dart	Initialized variables for statistic tracking	Cleaner structure
game_view_model.dart	Added statistics tracking per game: wins, losses, incorrect guesses per game	Separate calculations from UI
game_screen.dart	Displayed live stats above game grid	Enhance user experience
game_screen.dart	Added reset button for stats	Allow users to clear collected stats manually
game_screen.dart	Display loading indicator while loading	To fix not initialized error
game_screen.dart	Added an on-screen backspaces button by the side of the text field.	To improve user experience by making it easier to delete accidentally typed letters.

3. pubspec.yaml Changes

Package/ Asset	Why it was added
provider: ^6.1.5	To manage state following MVVM pattern

4. Additional File or Folders

File or Folder	Purpose and Justification
/model	Define core data structures that store game variables. Help to keep the logic clean and organized.
/view	holds all user interfaces files (game_screen.dart, home_screen.dart). It separates UI making updating and maintenance easier.
/view_model	Contains game_view_model.dart, which manages all game logic, keyboard colors, and statistics. Connect the UI and Data layers.

5. Reflective Summary

The refactored Nulldle app now follows best practices in Flutter mobile application development. Separating the UI and logic using the MVVM pattern and Provider for state management, the code has become cleaner, easier to maintain, and more scalable. Bug fixes improved gameplay accuracy, and new features like statistics tracking provide better feedback and experience to users. Improved input handling and UI updates make the app more user-friendly on mobile devices. Overall, the refactor and fixes for the application ensures functional correctness, good structure, and easier future development.